

Batch Norm vs Layer Norm, Worked Out by Hand

Same formula, different axis — and that axis is the whole story

Both normalisation layers run the *identical* two-line formula: standardise to zero mean and unit variance, then apply a learnable affine γ, β . The only thing that differs is *which axis you reduce over*. We take one tiny activation batch X of shape ($N=2, C=3$) and compute *both* norms on it by hand, then check every number against `torch.nn.BatchNorm1d` and `torch.nn.LayerNorm`. Population (biased) variance and $\varepsilon = 10^{-5}$ throughout, matching PyTorch.

What this document does. It fixes a 2×3 batch $X = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 6 & 9 \end{bmatrix}$ — two samples, three features — and standardises it twice. *BatchNorm* collapses each *column* (a feature, across the batch); *LayerNorm* collapses each *row* (a sample, across its features). Same $\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \varepsilon}$, opposite axis. We watch the two answers diverge and explain exactly why.

How to read this. Read Steps 1–3 to lock in the shared formula and the two reductions, then Step 4 for the side-by-side. The boxes flag where the choice of axis actually bites in practice (batch size 1, `model.eval()`, transformers vs CNNs). Every displayed number was produced by the reference program in Appendix A and verified against PyTorch to floating-point tolerance.

Concepts covered, in order: the shared standardise-then-affine formula · population variance and ε · BatchNorm as a per-feature column reduction · LayerNorm as a per-sample row reduction · the axis as the *only* difference · the affine γ, β · BatchNorm’s running EMA statistics and why `train` $\neq$ `eval` · why LayerNorm is identical in train and eval · which layer to reach for, and when.

1 The setup: one batch, normalised two ways

A `BatchNorm1d`/`LayerNorm` layer sees a matrix of activations with rows = samples in the batch and columns = features. We shrink to the smallest matrix that still distinguishes the two axes — $N = 2$ samples and $C = 3$ features — and pick numbers where the arithmetic is clean:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 6 & 9 \end{bmatrix} \quad \begin{array}{l} \text{row } n = \text{sample } n \ (N = 2) \\ \text{col } c = \text{feature } c \ (C = 3) \end{array}$$

Note row 1 is exactly $3 \times$ row 0 — a deliberate choice that makes `LayerNorm`’s behaviour pop out in Step 3.

Quantity	Symbol	Value here
Batch size (samples)	N	2
Feature count	C	3
Epsilon (numerical floor)	ε	10^{-5}
Initial scale	γ	1 (per channel)
Initial shift	β	0 (per channel)
Variance convention	—	population / biased (<code>ddof</code> = 0)

Intuition

Picture X as a spreadsheet: rows are examples, columns are features. **BatchNorm** asks each *column* “what is your mean and spread *across the batch*?” and flattens it — so feature c ends up zero-mean across the N samples. **LayerNorm** asks each *row* “what is your mean and spread *across your own features*?” and flattens that — so sample n ends up zero-mean across its C features. One reduces down columns, the other across rows. That single choice of axis is the entire difference between the two layers.

2 Step 1 — The one formula both layers run

Strip away the names and both layers are the same two lines. Given a set of values to be standardised together (the *reduction group*), compute its mean μ and *population* variance σ^2 , then

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} \quad y = \gamma \hat{x} + \beta$$

with population variance

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2 \quad (\text{divide by } m, \text{ not } m-1).$$

Here m is the size of the reduction group and $\varepsilon = 10^{-5}$ keeps the denominator away from zero. With the initial $\gamma = 1, \beta = 0$ we have $y = \hat{x}$, so the affine is invisible *until the network learns* otherwise; we add it back in Step 5.

Everything that follows is just: choose what goes in the reduction group.

- **BatchNorm:** the group is one *column* — the N samples of a fixed feature c . So $m = N = 2$, and there is a separate (μ_c, σ_c^2) per feature.
- **LayerNorm:** the group is one *row* — the C features of a fixed sample n . So $m = C = 3$, and there is a separate (μ_n, σ_n^2) per sample.

Watch out

PyTorch normalises with the *biased* (population) variance — divide by m — for both BatchNorm1d and LayerNorm in the forward pass. If you reproduce it with `np.var(..., ddof=1)` or `torch.var(...)` (which default to the *unbiased* $m - 1$ estimator) your $\hat{x}$ will be off by a factor $\sqrt{m/(m-1)}$. The one place $m - 1$ *does* appear is BatchNorm's running-variance buffer — Step 6.

3 Step 2 — BatchNorm: reduce down each column (per feature)

Fix a feature (column) and average over the $N = 2$ samples. Column means:

$$\mu_c = \frac{1}{N} \sum_n X_{n,c} \Rightarrow \mu_0 = \frac{1+3}{2} = 2, \quad \mu_1 = \frac{2+6}{2} = 4, \quad \mu_2 = \frac{3+9}{2} = 6,$$

so $\mu_c = (2, 4, 6)$. Column population variances:

$$\begin{aligned} \sigma_c^2 &= \frac{1}{N} \sum_n (X_{n,c} - \mu_c)^2 \Rightarrow \sigma_0^2 = \frac{(1-2)^2 + (3-2)^2}{2} = \frac{1+1}{2} = 1, \\ \sigma_1^2 &= \frac{(2-4)^2 + (6-4)^2}{2} = \frac{4+4}{2} = 4, \\ \sigma_2^2 &= \frac{(3-6)^2 + (9-6)^2}{2} = \frac{9+9}{2} = 9, \end{aligned}$$

so $\sigma_c^2 = (1, 4, 9)$ and $\sqrt{\sigma_c^2 + \varepsilon} = (1.000, 2.000, 3.000)$ (the ε is negligible here). Standardising each entry by *its column's* stats:

$$\hat{X}^{\text{BN}} = \begin{bmatrix} \frac{1-2}{1} & \frac{2-4}{2} & \frac{3-6}{3} \\ \frac{3-2}{1} & \frac{6-4}{2} & \frac{9-6}{3} \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ 1 & 1 & 1 \end{bmatrix}$$

Check the *columns*: each has mean $\frac{-1+1}{2} = 0$ and variance $\frac{1+1}{2} = 1$. BatchNorm's promise is “every feature is zero-mean, unit-variance *across the batch*” — and indeed the column means are $(0, 0, 0)$, column variances $(1, 1, 1)$. The *rows*, however, are untouched as a group: row 0 is $(-1, -1, -1)$, which does *not* have mean 0 across its features. That is LayerNorm's job, not BatchNorm's.

4 Step 3 — LayerNorm: reduce across each row (per sample)

Now fix a sample (row) and average over the $C = 3$ features. Row means:

$$\mu_n = \frac{1}{C} \sum_c X_{n,c} \Rightarrow \mu_0 = \frac{1+2+3}{3} = 2, \quad \mu_1 = \frac{3+6+9}{3} = 6.$$

Row population variances:

$$\begin{aligned} \sigma_n^2 &= \frac{1}{C} \sum_c (X_{n,c} - \mu_n)^2 \Rightarrow \sigma_0^2 = \frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3} = \frac{1+0+1}{3} = 0.667, \\ \sigma_1^2 &= \frac{(3-6)^2 + (6-6)^2 + (9-6)^2}{3} = \frac{9+0+9}{3} = 6.000, \end{aligned}$$

so $\sqrt{\sigma_0^2 + \varepsilon} = 0.817$ and $\sqrt{\sigma_1^2 + \varepsilon} = 2.449$. Standardising each entry by *its row's* stats:

$$\hat{X}^{\text{LN}} = \begin{bmatrix} \frac{1-2}{0.817} & \frac{2-2}{0.817} & \frac{3-2}{0.817} \\ \frac{3-6}{2.449} & \frac{6-6}{2.449} & \frac{9-6}{2.449} \end{bmatrix} = \begin{bmatrix} -1.225 & 0 & 1.225 \\ -1.225 & 0 & 1.225 \end{bmatrix}$$

Check the *rows*: each has mean $\frac{-1.225+0+1.225}{3} = 0$ and variance $\frac{1.225^2+0+1.225^2}{3} = 1$. LayerNorm's promise is “every sample is zero-mean, unit-variance *across its own features*.”

Intuition

Why are the two rows of $\hat{X}^{\text{LN}}$ *identical*? Because row 1 = 3 × row 0, and standardisation $(x - \mu)/\sigma$ is invariant to any positive rescaling of the whole row: scale a vector by 3 and both its mean and its standard deviation scale by 3 too, cancelling. LayerNorm sees only the *shape* of a sample's feature pattern, not its overall magnitude. BatchNorm, reducing the other way, kept the two rows different (−1 vs +1) — it is the magnitude across the batch that it cares about.

5 Step 4 — The only difference is the axis

Same X , same formula $\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \varepsilon}$, same ε . The reductions differ only in their axis, and that produces two genuinely different outputs:

	BatchNorm1d	LayerNorm
Reduce over	axis 0 (samples), per column	axis 1 (features), per row
Group size m	$N = 2$	$C = 3$
Means computed	$\mu_c = (2, 4, 6)$	$\mu_n = (2, 6)$
Variances computed	$\sigma_c^2 = (1, 4, 9)$	$\sigma_n^2 = (0.667, 6.000)$
$\sqrt{\sigma^2 + \varepsilon}$	(1.000, 2.000, 3.000)	(0.817, 2.449)
What ends up unit-var	each <i>column</i>	each <i>row</i>
Output $\hat{X}$	$\begin{bmatrix} -1 & -1 & -1 \\ 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} -1.225 & 0 & 1.225 \\ -1.225 & 0 & 1.225 \end{bmatrix}$

In code the difference is literally one argument: the axis you tell `mean/var` to collapse. BatchNorm keeps a (μ, σ^2) pair *per feature* (length $C = 3$); LayerNorm computes a (μ, σ^2) pair *per sample* (length $N = 2$) on the fly and keeps nothing. Both then apply a γ, β of length C .

When this is the right choice

BatchNorm for CNNs and MLPs with a decent batch size (say ≥ 16 -ish): the per-feature statistics are estimated over many samples, which is stable and acts as a mild regulariser. **LayerNorm** for transformers, RNNs, and any small/variable/size-1 batch: it is per-sample, so it needs no batch at all and is unaffected by how many sequences happen to be in the batch. **GroupNorm** is the go-to when you have CNN-style channels but tiny batches (e.g. high-resolution detection/segmentation), splitting channels into groups instead of relying on the batch.

6 Step 5 — The learnable affine γ, β

Both layers finish with $y = \gamma \hat{x} + \beta$, where γ, β are learnable vectors of length C (one scale and shift *per feature column*). At initialisation $\gamma = 1, \beta = 0$, so $y = \hat{x}$ and the tables above *are* the outputs. The affine lets the network undo the normalisation if that helps — e.g. recover a non-zero mean a downstream layer prefers.

Worked affine on the BatchNorm output. Suppose training has driven $\gamma = (2, 2, 2)$, $\beta = (1, 1, 1)$. Apply column-wise to $\hat{X}^{\text{BN}}$:

$$y = 2 \hat{X}^{\text{BN}} + 1 = \begin{bmatrix} 2(-1) + 1 & 2(-1) + 1 & 2(-1) + 1 \\ 2(1) + 1 & 2(1) + 1 & 2(1) + 1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ 3 & 3 & 3 \end{bmatrix}.$$

The standardisation still happened (the layer learned *what* mean and spread it wants); γ, β just relabel the normalised axis afterward. Crucially the *gradient-friendly* part — the centred, unit-scaled $\hat{x}$ — is computed first, regardless of γ, β .

7 Step 6 — Inference: BatchNorm’s split personality, LayerNorm’s calm

This is where the axis choice has its sharpest consequence.

LayerNorm computes μ_n, σ_n^2 from the *current input row* every time, in training and at inference alike. There is nothing to remember; the forward pass is byte-for-byte identical in `model.train()` and `model.eval()`.

BatchNorm cannot do this at test time: a single test sample has no batch to average over (its column “variance” over one sample is 0). So during training it keeps a running exponential moving average (EMA) of the batch statistics and uses *those* fixed numbers at eval. With PyTorch’s default momentum 0.1 and buffers initialised to `running_mean = 0`, `running_var = 1`, one training step on our X updates (note the running *variance* uses the *unbiased* batch variance $\hat{\sigma}_c^2 = \frac{N}{N-1} \sigma_c^2 = (2, 8, 18)$):

$$\begin{aligned} \text{running_mean} &\leftarrow (1 - 0.1) \cdot 0 + 0.1 \cdot \mu_c = 0.1 \cdot (2, 4, 6) = (0.2, 0.4, 0.6), \\ \text{running_var} &\leftarrow (1 - 0.1) \cdot 1 + 0.1 \cdot \hat{\sigma}_c^2 = 0.9 + 0.1 \cdot (2, 8, 18) = (1.1, 1.7, 2.7). \end{aligned}$$

At *eval*, BatchNorm standardises with these buffers, *not* the batch:

$$\hat{X}^{\text{BN,eval}} = \frac{X - \text{running_mean}}{\sqrt{\text{running_var} + \varepsilon}} = \begin{bmatrix} 0.763 & 1.227 & 1.461 \\ 2.670 & 4.295 & 5.112 \end{bmatrix},$$

which is *nothing like* the clean ± 1 training output — because after a single step the running stats are still mostly their 0/1 initial values. That train/eval gap is real and expected; it shrinks as the EMA warms up over many steps.

Watch out

Two classic BatchNorm bugs, both from the batch axis. **(1)** Forgetting `model.eval()` at test time: the layer then normalises each test batch by *its own* statistics instead of the running ones, so predictions wobble with batch composition and differ from training-time behaviour. **(2)** Batch size 1 (or a tiny/shifting batch): the per-feature variance over one sample is 0, so $\hat{x}$ blows up to $0/\sqrt{\varepsilon}$ or is undefined, and the running estimates become noisy. LayerNorm has neither failure mode — it never looks at the batch axis.

Mini-example — this operation in isolation

BatchNorm at batch size 1 on a single feature value $x = 5$: the group is just $\{5\}$, so $\mu = 5$, $\sigma^2 = 0$, and $\hat{x} = (5 - 5)/\sqrt{0 + 10^{-5}} = 0$ — the signal is annihilated. LayerNorm on the same lone sample with features $(1, 2, 3)$ still has a full group of $C = 3$ values to work with: $\mu = 2$, $\sigma^2 = 0.667$, $\hat{x} = (-1.225, 0, 1.225)$. Same formula; only LayerNorm has enough data in its reduction group when the batch is tiny.

8 At a glance

#	Property	BatchNorm1d	LayerNorm
1	Reduction axis	samples (axis 0)	features (axis 1)
2	Stats are per-	feature (C pairs)	sample (N pairs)
3	Group size m	$N = 2$	$C = 3$
4	Forward variance	biased ($\div m$)	biased ($\div m$)
5	Learnable γ, β	length C	length C
6	Keeps running stats?	yes (EMA buffers)	no
7	train vs eval	<i>different</i>	identical
8	Works at batch size 1?	no ($\sigma^2 = 0$)	yes
9	Typical home	CNNs, MLPs	transformers, RNNs

9 Equation cheat-sheet

Shared standardise	$\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \varepsilon}$
Shared affine	$y = \gamma \hat{x} + \beta$ (init $\gamma=1, \beta=0$)
Population variance	$\sigma^2 = \frac{1}{m} \sum_i (x_i - \mu)^2$ (forward: $\div m$)
BatchNorm group	one column: $m = N$ samples; stats per feature c
LayerNorm group	one row: $m = C$ features; stats per sample n
BN running mean	$\leftarrow (1 - \rho) r_\mu + \rho \mu_{\text{batch}}$
BN running var	$\leftarrow (1 - \rho) r_v + \rho \hat{\sigma}_{\text{batch}}^2$ ($\hat{\sigma}^2$ unbiased, $\div(m-1)$)
BN eval	$\hat{x} = (x - r_\mu)/\sqrt{r_v + \varepsilon}$ (needs <code>model.eval()</code>)
LN train = eval	always; no buffers, per-sample stats

Appendix A Reference implementation

Every number above is reproducible from the following program, and is checked against `torch.nn.BatchNorm1d` and `torch.nn.LayerNorm` to floating-point tolerance.

```
import numpy as np, torch

eps = 1e-5
X = np.array([[1., 2., 3.],
              [3., 6., 9.]])          # (N=2 samples, C=3 features)
N, C = X.shape

# ---- BatchNorm: reduce down columns (over the N samples), per feature ----
mu_c = X.mean(axis=0)                 # (2, 4, 6)
var_c = X.var(axis=0)                 # population/biased: (1, 4, 9)
xhat_bn = (X - mu_c) / np.sqrt(var_c + eps)
print(xhat_bn)                       # [[-1 -1 -1] [ 1  1  1]]

# ---- LayerNorm: reduce across rows (over the C features), per sample ----
mu_n = X.mean(axis=1)                 # (2, 6)
var_n = X.var(axis=1)                 # population/biased: (0.667, 6.0)
xhat_ln = (X - mu_n[:, None]) / np.sqrt(var_n[:, None] + eps)
print(xhat_ln)                       # [[-1.225 0 1.225] [-1.225 0 1.225]]

# ---- verify against torch (both use biased var in forward) ----
Xt = torch.tensor(X)
bn = torch.nn.BatchNorm1d(C, eps=eps, dtype=torch.float64); bn.train()
ln = torch.nn.LayerNorm(C, eps=eps, dtype=torch.float64)
assert np.allclose(bn(Xt).detach().numpy(), xhat_bn)
assert np.allclose(ln(Xt).detach().numpy(), xhat_ln)

# ---- BN running stats (EMA), momentum=0.1; running_var uses UNBIASED var ----
mom = 0.1
run_mean = (1-mom)*0.0 + mom*mu_c     # (0.2, 0.4, 0.6)
run_var = (1-mom)*1.0 + mom*X.var(axis=0, ddof=1) # (1.1, 1.7, 2.7)
print(run_mean, run_var)
assert np.allclose(bn.running_mean.detach().numpy(), run_mean)
assert np.allclose(bn.running_var.detach().numpy(), run_var)

# ---- BN eval uses running stats (train != eval); LN is identical ----
bn.eval()
print(bn(Xt).detach().numpy())         # [[0.763 1.227 1.461] [2.67 4.295 5.112]]
ln.eval()
assert np.allclose(ln(Xt).detach().numpy(), xhat_ln) # LN unchanged

# ---- learnable affine example: gamma=2, beta=1 ----
y = 2*xhat_bn + 1
print(y)                               # [[-1 -1 -1] [ 3  3  3]]
```

— end of document —